

Ejercicio Práctico 1

Paso 1: Crear el archivo HTML

Abre tu editor de texto y crea un nuevo archivo llamado EP2.html. Incluye los elementos básicos de la estructura HTML:

```
HTML

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejercicio Práctico 2</title>
</head>
<body>
  <script src="app.js"></script>
</body>
</html>
```

Estructura de archivos en el explorador:

- **Carpeta del proyecto**
 - EP2.html
 - app.js (se creará en el siguiente paso)

Paso 2: Crear el archivo JavaScript

En la misma carpeta del proyecto, crea un archivo llamado app.js e inserta el siguiente código:

```
JavaScript

// Este es un método para devolver un cuadro de alerta
window.alert("¡Hola mundo!");
```

Análisis del código en app.js:

- **Línea 1:** Comentario explicativo.
- **Línea 2:** Método window.alert() que muestra el mensaje.

Paso 3: Probar en el navegador

1. **Guarda** todos los cambios en ambos archivos.
2. **Abre** EP2.html en tu navegador preferido.
3. **Resultado:** Deberías ver un cuadro de alerta con el mensaje **¡Hola Mundo!**.

[!IMPORTANT] **Solución de problemas:** Si la alerta no aparece, revisa que el nombre del archivo app.js coincida exactamente con la referencia en la etiqueta <script> y que ambos archivos estén en la misma carpeta.

¿Te gustaría que añada una breve explicación sobre cómo funcionan las rutas relativas para evitar errores de carga en el archivo JS?

Ejercicio Práctico 2: Juego de Piedra, Papel o Tijera

Paso 1: Definición del Desafío

Usando todo lo visto en esta lección, realiza en código JavaScript un juego de **piedra, papel o tijera** de dos jugadores.

Paso 2: Requisitos del Sistema

Al finalizar cada jugada, el programa deberá imprimir por pantalla las estadísticas detalladas de la sesión:

- **Rondas ganadas** para el jugador 1.
- **Rondas ganadas** para el jugador 2.
- **Cantidad de empates** registrados.

Paso 3: Estructura Sugerida del Código

JavaScript

```
// 1. Declaración de variables para contadores
let ganadasJ1 = 0;
let ganadasJ2 = 0;
let empates = 0;

// 2. Lógica de captura de jugadas (Ejemplo: prompt)
// 3. Comparación de resultados
// 4. Impresión de estadísticas por pantalla (Ejemplo: console.log o alert)
```

Ejemplo de salida esperada en consola:

"Estadísticas: Jugador 1: 2 | Jugador 2: 1 | Empates: 0"

Ejercicio Práctico 3: Cálculo del Área de un Círculo

Descripción de la Actividad

En esta actividad aplicarás conceptos de JavaScript y manipulación del **DOM** para resolver un problema matemático. El objetivo es interactuar con el usuario mediante cuadros de diálogo, realizar operaciones con el objeto Math y mostrar resultados en diversos canales de salida.

Conceptos a consolidar:

- `prompt()`
 - `console.log()`
 - `window.alert()`
 - `document.getElementById().innerHTML`
-

Instrucciones de la Tarea

Debes desarrollar una aplicación web que calcule el área de un círculo aplicando la fórmula:

$$Área = \pi \times r^2$$

(Donde el radio r se obtiene a partir del diámetro ingresado por el usuario).

Paso 1: Estructura HTML (index.html)

Crea un archivo con la estructura básica de un documento HTML e incluye un elemento `<div>` con un **id** específico para mostrar el resultado final.

Paso 2: Lógica en JavaScript (app.js)

El script debe realizar las siguientes acciones:

1. **Solicitar** al usuario el diámetro del círculo mediante `prompt()`.
2. **Calcular** el radio (diámetro dividido entre 2).
3. **Obtener** el área utilizando `Math.PI` y `Math.pow(radio, 2)`.
4. **Mostrar** el resultado a través de:
 - La consola (`console.log()`).
 - Una ventana emergente (`window.alert()`).
 - La página web (`document.getElementById().innerHTML`).

Paso 3: Vinculación

Asegúrate de vincular el archivo `app.js` a tu `index.html` usando la etiqueta `<script src="app.js"></script>` antes del cierre del `body`.

[!TIP]

Nota técnica: Recuerda que los valores obtenidos mediante `prompt()` son cadenas de texto; asegúrate de tratarlos como números para realizar los cálculos correctamente.

EJERCICIO 4: Días del Mes con Arreglos

Descripción del Desafío

Realiza un programa donde el usuario introduzca el número de un mes (**1 a 12**) y la respuesta del sistema sea la cantidad de días que tiene dicho mes.

Requerimientos Técnicos

Para la resolución de este ejercicio, debes cumplir con los siguientes puntos:

- **Uso de Arreglos:** Utiliza arreglos (arrays) para almacenar la información de los meses y sus respectivos días.
 - **Lógica Condicional:** Realiza todas las condicionales que estipules necesarias para procesar la entrada del usuario.
 - **Validación de Datos:** Verifica todas las posibles entradas de datos (asegúrate de que el usuario ingrese un número válido dentro del rango permitido).
-

Guía de Implementación sugerida

1. **Declarar el arreglo** con la cantidad de días de cada mes (considerando febrero con 28 días).
2. **Capturar la entrada** del usuario mediante un `prompt()`.
3. **Validar** que el dato sea un número entre 1 y 12.
4. **Acceder al índice** correspondiente del arreglo y mostrar el resultado.

EJERCICIO 5: Identificación del Mayor y Menor con Ordenamiento Burbuja

Objetivo de la Actividad

Desarrollar un programa en JavaScript que procese tres valores ingresados por el usuario para determinar el número mayor y el menor, manejando casos de igualdad. Este ejercicio refuerza el uso de **arrays**, **ciclos (for, do-while)**, **condicionales** y el algoritmo de **ordenamiento burbuja**.

Nota Teórica: El ordenamiento burbuja es un algoritmo simple que funciona comparando elementos adyacentes e intercambiándolos si están en el orden incorrecto, repitiendo el proceso hasta que la lista esté ordenada.

Instrucciones de la Tarea

1. **Estructura del Proyecto:**
 - Crea un archivo index.html con la estructura básica.
 - Crea un archivo app.js y vincúlalo usando `<script src="app.js"></script>`.
2. **Captura de Datos:**
 - Solicita al usuario ingresar **3 números** mediante `prompt()`.
 - Convierte las entradas a números flotantes usando `parseFloat()`.
 - Almacena los valores en un arreglo utilizando el método `.push()`.
3. **Lógica de Ordenamiento:**
 - Implementa el **ordenamiento burbuja** utilizando un ciclo `do-while`.
 - Compara elementos consecutivos y realiza los intercambios necesarios hasta que el arreglo quede ordenado de menor a mayor.
4. **Identificación de Valores:**
 - **Menor valor:** Se encontrará en la posición `array[0]`.
 - **Mayor valor:** Se encontrará en la posición `array[array.length - 1]`.
5. **Validación Especial:**
 - Si el valor menor y el mayor son iguales, muestra un mensaje indicando que los tres números ingresados son **idénticos**.
6. **Salida de Resultados:**
 - Muestra el resultado final al usuario mediante `document.write()` o `alert()`.

EJERCICIO 6: Función de Cálculo de Monto con IVA y Descuento

Descripción del Desafío

Realiza una función en JavaScript que permita procesar un pago aplicando impuestos y beneficios comerciales. La función debe recibir por parámetro un **monto** y un **descuento**.

Requerimientos de la Función

El desarrollo debe cumplir con los siguientes procesos lógicos:

- **Cálculo de IVA:** Realiza los cálculos necesarios para agregar el IVA al monto inicial.
- **Aplicación de Descuento:** Aplica el porcentaje de descuento indicado al monto.
- **Retorno de Datos:** La función debe retornar el **valor resultante** final tras ambas operaciones.

Especificaciones Técnicas Sugeridas

1. **Definición:** Utiliza una función con nombre descriptivo (ej. `calcularTotal`).
2. **Parámetros:** Asegúrate de recibir dos valores numéricos.
3. **Operación:**
 - Suma el IVA (ej. 19% o 21% según corresponda).
 - Resta el valor del descuento.
4. **Resultado:** Devuelve el total usando la palabra clave `return`.

EJERCICIO 7: Adivina el Número (1–10)

Objetivo y Contexto

Desarrollar un mini-juego interactivo donde el computador selecciona un número aleatorio y el usuario debe adivinarlo en un máximo de **3 intentos efectivos**. Este ejercicio integra funciones, validación de rangos, generación de números aleatorios y manipulación de arreglos para el control de historial.

Especificaciones Técnicas

1. Estructura del Proyecto

- index.html: Estructura base con un `<div id="historial">` para mostrar los intentos.
- app.js: Lógica completa del juego vinculada mediante `<script>`.

2. Generación del Número Secreto

Utiliza las funciones matemáticas de JavaScript para obtener un entero entre 1 y 10:

```
JavaScript   
  
const secreto = Math.floor(Math.random() * 10) + 1;
```

3. Reglas y Validaciones

El flujo debe respetar las siguientes condiciones de control:

- **Límite de intentos:** Máximo 3 oportunidades.
- **Rango permitido:** Solo valores del 1 al 10. Si el usuario ingresa un dato fuera de rango, se debe repetir la solicitud sin descontar el intento.
- **Prevención de repetidos:** Si el usuario ingresa un número que ya intentó anteriormente, el programa debe advertirle y permitirle reintentar sin gastar sus oportunidades.

4. Función de Control (yaUsado)

Implementa una función dedicada para verificar el historial:

```
JavaScript   
  
function yaUsado(numero, lista) {  
    return lista.includes(numero);  
}
```

Guía de Implementación (Flujo sugerido)

1. **Inicialización:** Crea un arreglo vacío `usados = []` y un contador de intentos.
2. **Ciclo de Juego:** Mientras queden intentos disponibles:
 - Captura el dato con `prompt()` y conviértelo con `parseInt()`.
 - **Validar Rango:** `if (isNaN(n) || n < 1 || n > 10)`.
 - **Validar Duplicados:** Usar la función `yaUsado()`.
 - Si pasa las validaciones, agregar al arreglo `usados` y comparar con el secreto.
3. **Resultados:**
 - **Acierto:** Mostrar mensaje de victoria y finalizar.
 - **Fallo:** Informar los intentos restantes.
 - **Derrota:** Si agota los 3 intentos, revelar el número secreto.

4. **Actualización del DOM:** Reflejar el historial en la página:
`document.getElementById('historial').innerHTML = "Intentos: " + usados.join(', ');`
-

Criterios de Evaluación

- Correcta implementación de `Math.random()` y `Math.floor()`.
- Validación efectiva de entrada (1..10) y números repetidos.
- Manejo preciso del límite de 3 intentos.
- Mensajería clara al usuario (alertas de error, éxito o fin de juego).

EJERCICIO 08: Identificador de Llamadas con Objetos

Descripción del Desafío

Un teléfono celular cuenta con un identificador de llamadas. Esta funcionalidad permite que, al ingresar una llamada, el propietario sepa quién llama si ese número se encuentra previamente registrado en su agenda.

El objetivo es realizar un programa en JavaScript que imite esta funcionalidad: dado un número de teléfono, buscarlo en una estructura de datos y mostrar a quién pertenece.

Requerimientos Técnicos

Para resolver este ejercicio, el programa debe integrar los siguientes elementos:

- **Estructura de Objetos:** Define una agenda utilizando objetos que vinculen nombres con sus respectivos números telefónicos.
 - **Lógica de Búsqueda:** Implementa un método que recorra la estructura para comparar el número ingresado con los existentes.
 - **Interfaz de Usuario:** * El sistema debe recibir un número como entrada.
 - Si el número existe, muestra el nombre del contacto.
 - Si el número no existe, muestra un mensaje indicando que el contacto es desconocido.
-

Guía de Implementación sugerida

1. **Crear la Agenda:** Utiliza un arreglo de objetos (ej. `const agenda = [{nombre: "Ana", tel: "12345"}, ...]`).
2. **Capturar la Llamada:** Solicita el número mediante un `prompt()`.
3. **Procesar la Búsqueda:** Utiliza métodos de arreglos como `.find()` o un ciclo `for...of` para localizar el contacto.
4. **Mostrar Resultado:** Informar al usuario el nombre encontrado o la ausencia del mismo.

EJERCICIO 09: Conteo de Vocales en Palabras

Introducción

En este ejercicio aplicarás funciones en JavaScript para procesar cadenas y arreglos. La actividad te permitirá afianzar conceptos como **funciones expresivas**, bucles de iteración y métodos esenciales como `toLowerCase()`, `includes()` y `join()`.

Objetivo

Desarrollar un programa que, a partir de palabras ingresadas por el usuario, cuente la cantidad total de vocales presentes en el conjunto, utilizando lógica de programación y manipulación de estructuras de datos.

Descripción de la Tarea

1. Preparación del Entorno

- Crea un archivo `index.html` con la estructura básica.
- Crea el archivo `app.js` y enlázalo mediante `<script src="app.js"></script>`.

2. Ingreso de Datos

- Solicita al usuario la cantidad de palabras que desea ingresar mediante `prompt()`.
- Captura cada palabra y almacénalas dentro de un **array**.

3. Función de Conteo (Lógica Interna)

Define una **función expresiva** llamada `contarVocales(palabra)` que realice lo siguiente:

- Convierta el texto a minúsculas con `toLowerCase()`.
- Recorra cada carácter y verifique si es una vocal utilizando un arreglo de referencia y el método `includes()`.

4. Procesamiento Total

- Combina todos los elementos del array de palabras en una sola cadena de texto usando `join("")`.
- Aplica la función de conteo sobre la cadena resultante para obtener el gran total.

5. Salida de Resultados

Muestra el conteo final a través de tres canales:

1. **Consola:** `console.log()`.
 2. **Ventana emergente:** `window.alert()`.
 3. **Interfaz Web:** `document.getElementById("resultado").innerHTML`.
-

Información de Entrega

- **Tiempo estimado:** 40 a 60 minutos.
- **Formato:** Archivo comprimido (.zip) que contenga el HTML y el JS.
- **Opcional:** Capturas de pantalla del programa en ejecución.